Unix for Software Developers

The Shell

Benedict Reuschling

November 11, 2024



Overview

The shell serves as the command interpreter of the Unix system. The different shells differ in usage but not in their underlying concepts. We discuss the most common shell features.

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Shell Basics

The shell is the main interface between the user and the Unix system. The shell wraps around the inner core like a sea shell. The kernel represents the core. The user can not directly interact with the kernel, but has to use the shell for that. The shell takes care of syntax checking so that no invalid or erroneous commands reach the kernel.
Shells have similar features to make them easier to use or to type less.

Overview

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Command Completion

A common feature which is available in shells is command completion. It allows typing the first characters of a command and the shell completes the rest of the command by pressing ⇆ (tabulator). This is helpful with long filenames or complicated, similar sounding commands.

Example:

```
$ ls -l
total 0
-rw-r--r--  1 benedict  wheel  0 Aug 26 15:32 Verylongfilename
```

By entering V followed by ⇆ the complete filename appears on the prompt, because there is one file.

Command Completion

Completion of commands happens as far as the name of the command or the file is unambiguous.

Example:

```
$ ls -l
total 0
-rw-r--r--  1 benedict  wheel  0 Aug 26 15:39 BillsPaid2018
-rw-r--r--  1 benedict  wheel  0 Aug 26 15:39 BillsPaid2019
```

When trying to list the `BillsPaid2018` file, one can enter `ls -l B` ⇤⇥. The shell will complete the word until `BillsPaid201`, the rest is ambiguous. Either an 8 or 9 follows. Entering the next character (5 in this case) and hitting ⇤⇥ again, the shell will complete the rest of the file to `BillsPaid2018`. Often, shells will give a visual (in form of a short screen flash) or audible feedback from the PC speakers when the user needs to enter the next characters to make command completion unambiguous again.

Overview

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Moving and Erasing Around the Cursor Position

The Unix shell provides a couple of handy defaults for moving the cursor around on long commands with parameters. Instead of moving from character to character using the arrow keys, moving the cursor to the beginning of the line using Ctrl + A is much faster. Similarly, placing the cursor after a word and hitting Ctrl + W will delete the word as a whole.

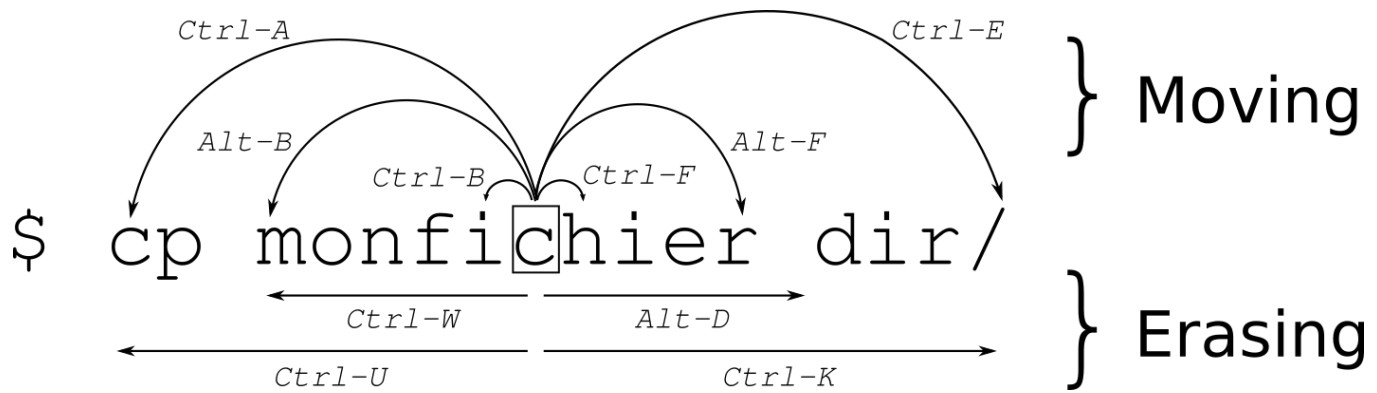


Figure: Shell shortcuts for moving and erasing around the cursor position
https://clementc.github.io/blog/2018/01/25/moving_cli/

Overview

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Environment Variables

Every shell sets and accesses so called environment variables. Their purpose is to store common information that programs use without having to store them separately for each. Configuring applications without having to make changes to the program's source code is possible with this data. This will be helpful later when programming the shell to store more information for each individual user (for example the name of the user for a personal greeting when logging in).

Commonly Used Environment Variables

The following variables often exist:

`USER`	The name of the user logged in
`PATH`	List of directories to search for executables, separated by a colon (:).
`DISPLAY`	Name of the X11 screen to send graphical output to.
`SHELL`	The used shell
`TERM`	The users terminal. Used to determine the terminal's capabilities.
`OSTYPE`	Operating system type, for example FreeBSD.
`MACHTYPE`	The CPU architecture of the system.
`EDITOR`	Text editor used by default (set to `vi` most of the time).
`PAGER`	The preferred program to view long texts.
`MANPATH`	List of colon separated directories where to search manual pages.

Echoing and Using Environment Variables

The `set` command prints all current environment variables to the screen. Display the value of a variable by using `echo`, followed by a `$` and then the name of the variable.

```
$ echo $EDITOR
vi
$
```

With this mechanism, one can use the values of the commands directly. For example, instead of using `vi newfile` use `$EDITOR newfile`, when `$EDITOR` is set to an installed editor.

Setting Environment Variables

Creating new environment variables for the shell session. They remain set until closing the shell or logging out. Next time when the user logs into the system they are unset again. The chapter about shell scripts describes how to persist variables.
When setting a new variable, one should check with `echo` whether such a variable already exists. This prevents accidental overwrites of important variables.
To set a variable, enter a name for it (capital letter are standard) directly followed by = and the value which the variable should get.

```
$ echo $TSHIRTSIZE

$ TSHIRTSIZE=XL
$ echo $TSHIRTSIZE
XL
$
```

Creating empty variables is also possible. Set their values later using the same syntax as shown above.

Overview

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Command History

Commands like `ls` are often entered. When there is a long list of parameters, it becomes tedious to enter them all over again. To solve this, the shell provides a history function. The shell saves every accepted shell command for later retrieval. By pressing [↑] one can cycle through previous commands and print it to the shell prompt. This way, find the one to use and hit [↵]. To go forward in the history, use [↓].

Searching in Command History

Many shells keep a record of commands entered in a history file (`.histfile`). This way, the user does not have to enter the command again, but can pull it back from history to save typing. To access the shell history search, press Ctrl+r. The prompt changes to `bck-i-search:`, indicating that it can search back through the history in a case insensitive manner (to increase the chance of matching something). Start typing the command and any matching entries will be written to the current prompt. Pressing Ctrl+r repeatedly will cycle through more matches if there are any or until the beginning of the history file is reached (since we're searching backwards). Press Esc or Ctrl+c to exit. When an entry is found, press Enter to execute that command.

Fuzzy History Searches using fzf

Fuzzy search allows searching for portions of the word at any position in the command to be found. By installing a package called `command-line fuzzy finder`[1] (fuzzy search) and `source`-ing two shell configuration files that come with it (`.zsh` for zsh or `.bash` for the bash), the functionality is activated. On FreeBSD these files are located in `/usr/local/share/examples/fzf/shell/` and are called `key-bindings.zsh` and `completion.zsh`, respectively. The exact locations are dependent of which Unix system is used. Running a `find /usr/ -name "fzf*"` should list them.

When pressing Ctrl + r now, the screen will be split and the more characters are entered, more search results will be displayed in the upper part. Navigate to the results with the arrow keys and press Enter to chose one. The search also includes characters that match anywhere within a long command (even the arguments). The longer the history file becomes over time, the better the search results will be. A lot of commands are entered frequently and by calling them up from history, entering them is not only faster, but also prevents typos from not having to re-enter them again.

[1] Homepage: `https://github.com/junegunn/fzf`

Overview

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Input and Output Redirection

A Unix program input from the keyboard (`stdin`) and outputs results on the screen (`stdout`).
The shell allows redirecting stdin and stdout.

```
$ ls
C_Programming ShellProgramming Shell tmp
$
$ ls > dat1
$ cat dat1
C_Programming ShellProgramming Shell tmp dat1
$
```

Redirecting the output of `ls` into the file `dat1`. The character to instruct the shell to do this is >.

Input and Output Redirection

```
$ echo ls -l > proc
$ ls -l
total 10
drwxr-xr-x 6 bcr staff 512 Jan  4 16:31 C_Programming
drwxr-xr-x 2 bcr staff 512 Jan  7 17:17 Shell
drwxr-xr-x 3 bcr staff 512 Dec 11 11:41 ShellProgramming
-rw-r--r-- 1 bcr staff   6 Jan  7 17:21 proc
drwxr-xr-x 3 bcr staff 512 Jan  7 17:16 tmp
$ cat proc
ls -l
$ sh < proc
total 10
drwxr-xr-x 6 bcr staff 512 Jan  4 16:31 C_Programming
drwxr-xr-x 2 bcr staff 512 Jan  7 17:17 Shell
drwxr-xr-x 3 bcr staff 512 Dec 11 11:41 ShellProgramming
-rw-r--r-- 1 bcr staff   6 Jan  7 17:21 proc
drwxr-xr-x 3 bcr staff 512 Jan  7 17:16 tmp
```

The shell created the file `proc` containing the string `ls -l`. Next, redirecting the input for `sh` so that `sh` reads its instructions from `proc` and executes them.

Input and Output Redirection

Combining these two mechanisms also works: input and output redirection.

```
$ sh < proc > dat2
$ cat dat2
total 10
drwxr-xr-x 6 bcr staff 512 Jan  4 16:31 C_Programming
drwxr-xr-x 2 bcr staff 512 Jan  7 17:17 Shell
drwxr-xr-x 3 bcr staff 512 Dec 11 11:41 ShellProgramming
-rw-r--r-- 1 bcr staff   0 Jan  7 17:25 dat2
-rw-r--r-- 1 bcr staff   6 Jan  7 17:21 proc
drwxr-xr-x 3 bcr staff 512 Jan  7 17:16 tmp
```

Input and Output Redirection

Use >> to redirect the output to the end of a file.

```
$ echo Another line at the end >> dat2
$ cat dat2
total 10
drwxr-xr-x 6 bcr staff 512 Jan  4 16:31 C_Programming
drwxr-xr-x 2 bcr staff 512 Jan  7 17:17 Shell
drwxr-xr-x 3 bcr staff 512 Dec 11 11:41 ShellProgramming
-rw-r--r-- 1 bcr staff   0 Jan  7 17:25 dat2
-rw-r--r-- 1 bcr staff   6 Jan  7 17:21 proc
drwxr-xr-x 3 bcr staff 512 Jan  7 17:16 tmp
Another line at the end
```

Input and Output Redirection

```
$ ls xyz > dat3
xyz: No such file or directory
$ ls -l
total 10
drwxr-xr-x 6 bcr staff 512 Jan  4 16:31 C_Programming
drwxr-xr-x 2 bcr staff 512 Jan  7 17:17 Shell
drwxr-xr-x 3 bcr staff 512 Dec 11 11:41 ShellProgramming
-rw-r--r-- 1 bcr staff 418 Jan  7 17:25 dat2
-rw-r--r-- 1 bcr staff   0 Jan  7 17:25 dat3
-rw-r--r-- 1 bcr staff   6 Jan  7 17:21 proc
drwxr-xr-x 3 bcr staff 512 Jan  7 17:16 tmp
```

Error output is not redirected. dat3 exists, but is empty (filesize is 0).

Summary

Redirect command input (stdin) and output (stdout) using <, > and >>. Error messages (stderr) are not redirected.

Overview

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Combining Commands

Separate commands on one line using ; (semicolon) and grouped together using (and).

```
$ date; ls -l
Mon Jan 7 17:42:47 MET 2002
total 12
-rw-r--r-- 1 bcr staff 418 Jan  7 17:25 dat2
-rw-r--r-- 1 bcr staff   0 Jan  7 17:25 dat3
-rw-r--r-- 1 bcr staff   6 Jan  7 17:21 proc
drwxr-xr-x 3 bcr staff 512 Jan  7 17:16 tmp
```

```
$ (date; ls -l) > dat
Mon Jan 7 17:45:47 MET 2002
total 12
-rw-r--r-- 1 bcr staff 418 Jan  7 17:25 dat2
-rw-r--r-- 1 bcr staff   0 Jan  7 17:25 dat3
-rw-r--r-- 1 bcr staff   6 Jan  7 17:21 proc
drwxr-xr-x 3 bcr staff 512 Jan  7 17:16 tmp
$
```

Combining Commands - Boolean Logic

While the ; (Semikolon) is interpreted as a logical OR, two && between commands are used as a logical AND. The second command only executes when the first command did not return an error.

```
$ (date && ls -l) > dat
Mon Jan 7 17:45:47 MET 2002
total 12
-rw-r--r-- 1 bcr staff 418 Jan  7 17:25 dat2
-rw-r--r-- 1 bcr staff   0 Jan  7 17:25 dat3
-rw-r--r-- 1 bcr staff   6 Jan  7 17:21 proc
drwxr-xr-x 3 bcr staff 512 Jan  7 17:16 tmp
$
```

If `date` would fail for some reason, the `ls -l` is not executed and as a result, `dat` is empty (no output written to it). For example, renaming `date` to a nonexisting command like `data` will cause such a failure.

```
$ data && echo "Output"
Command not found: data
$
```

Aliases

To make typing long and complex commands easier, use aliases as a shortcut. The alias is a placeholder for the actual command, which the shell expands when it is encountered. Using the `alias` keyword, abbreviations are defined on the shell ad-hoc for the current session. When used in a shell configuration file, they become active as soon the shell is started.
A common alias is ll for `ls -l`:

```
$ alias ll='ls -l'
```

Make sure to not leave any spaces around the = sign. From now on, typing ll will execute `ls -l`.
Use the `type` command to see if a command is an alias definition and what it resolves to.

```
$ type ll
ll is an alias for ls -l
```

Be careful not to overwrite existing commands with an alias!

Overview

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Background Jobs

Under normal circumstances the shell executes a command and releases control of the prompt back to the user after it has finished.
A program runs in the background by appending & (ampersand) to the end of the command line.

```
$ find / -name "*.c" -print > find.txt &
[1] 4106
$
```

The shell returns the process ID (called PID, 4106 in this case). This PID uniquely identifies the process.
Display running processes of the user by running ps.

```
$ ps
PID  TTY      TIME CMD
4026 pts/5    0:00 ksh
4109 pts/5    0:01 find
4037 pts/5    0:00 bash
4111 pts/5    0:00 ps
$
```

Background Jobs

Send a background job a signal to abort using `kill`.

```
$ sleep 10000 &
[2] 4116
$ ps
PID  TTY     TIME CMD
4026 pts/5   0:00 ksh
4109 pts/5   0:01 find
4037 pts/5   0:00 bash
4116 pts/5   0:00 sleep
4111 pts/5   0:00 ps
$ kill 4116
Terminated    sleep 10000
$ ps
PID  TTY     TIME CMD
4026 pts/5   0:00 ksh
4109 pts/5   0:01 find
4118 pts/5   0:00 ps
4037 pts/5   0:00 bash
```

When a user logs off, all background processes that are still running are "killed" automatically by the system.

Background Jobs

When the background job should continue to run even after the user has logged off, use `nohup` (**no h**ang **up**).

```
$ nohup grep "<math.h>" *.c > use_mathlib &
[1] 4122
$
$ exit
login: _
```

When a program runs in the background, the user can bring it to the front again using `fg` (**f**ore**g**round) and back again with `bg` (**b**ack**g**round) using so called suspension characters.
Here we background a process and bring it back to the front again:

```
$ sleep 60&
[1] 7163
$ fg
sleep 60
```

Background Jobs

When background processes are running, produce a list using `jobs`:

```
$ sleep 70&
[1] 7166
$ sleep 80&
[2] 7167
$ jobs
[1]- Running sleep 70 &
[2]+ Running sleep 80 &
$
```

`fg` (engl. **f**ore**g**round) can take the job number as a parameter (with a % in front) to specify which job should run in foreground mode:

```
$ jobs
[1]- Running sleep 70 &
[2]+ Running sleep 80 &
$ fg %2
sleep 80
```

Background Jobs

`fg` causes a resurrection to the screen until the program finishes. The key combination ^Z (Ctrl + Z) stops the program and `bg` backgrounds it.

```
$ sleep 100&
[1] 7174
$ fg
sleep 100
^Z
[1]+ Stopped sleep 100
$ jobs
[1]+ Stopped sleep 100
$ bg
[1]+ sleep 100 &
$ jobs
[1]+ Running sleep 100
$
```

Define the suspend character ^Z using stty (`stty susp`).

Overview

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Wildcards

Using the characters *, ?, and brackets [], writing filenames is easier.
Expand such wildcards *before* the shell executes the command with the expanded strings.
Let's say we have the following files in a directory:

```
$ ls
123.x      abc1       xy1        xy2345     xywait
abc        xy         xy2        xy2345.c
$
```

An asterisk (*) represents characters of 0 or more characters without the colon (.) at the beginning (hidden files).
Listing all files that start with xy:

```
$ ls xy*
xy         xy1        xy2     xy2345     xy2345.c   xywait
$
```

Wildcards

List all files:

```
$ ls *
123.x       abc1        xy1         xy2345      xywait
abc         xy          xy2         xy2345.c
$
```

List all C source code files (ending in .c):

```
$ ls *.c
xy2345.c
$
```

List all files that contain a b in their filename:

```
$ ls *b*
abc    abc1
$
```

Wildcards

A question mark (?) represents any single character.
List all files that start with xy and whose name is three 3 characters long.

```
$ ls xy?
xy1  xy2
$
```

The question mark is often used to select files that contain non-printable characters. Perhaps someone used the `mv` (move) command and made a typo which resulted in a non-printable character. Now this character is part of the filename.

Wildcards

Creating the file:

```
$ echo Test with a non-printable character > trial
$
```

When moving that file, the error occurs:

```
$ mv trial trial^G
$ ls
123.x      abc1       xy         xy2    xy2345.c
abc        trial      xy1        xy2345 xywait
$
```

Use the ? to correct the filename:

```
$ mv trial? trial1
$ ls
123.x      abc1       xy         xy2    xy2345.c
abc        trial1     xy1        xy2345 xywait
$
```

Wildcards

A string in brackets represents one of the characters within the brackets (selection). List all files that start with xy, followed by a number:

```
$ ls xy[0123456789]
xy1   xy2
$
```

List all files that start with xy, followed by a number and then any (even an empty) string of characters.

```
$ ls xy[0-9]*
xy1         xy2         xy2345      xy2345.c
$
```

Caution: The * combined with some commands (for example `rm`) can have unintended consequences:

```
$ pwd
/home/bcr/researchresults
$ rm -r *
```

This removed all files and directories and those below it!

Overview

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Quoting

The characters [,], *, ?, <, >, \, and & are so called meta characters. Used literally prepend the character \. \ removes the special meaning of the following character for the shell.

```
$ echo Test > \>
$ ls \>
>
$ cat \>
Test
$
```

Recommended when quoting single characters.

Quoting

When preserving the meaning of a string, then enclose the string in ".

```
$ echo Test > '>>>>>>abc>>>>>>'
$ echo Test > ">>>>>>123>>>>>>"
$ ls \>*
>                  >>>>>>123>>>>>>  >>>>>>abc>>>>>>
$
```

A string in single quotes always preserves it's meaning, which means that the shell reads from the first ' character until the next '.
Within a string enclosed in " the shell interprets the characters $, ' and ". Showing an example later.

Quoting

The command

```
grep pattern file
```

searches `file` for lines containing *pattern*.

```
$ cat trial
This line contains a * character
this line does not
$
$ grep line trial
This line contains a * character
this line does not
$
$ grep "*" trial
This line contains a * character
$
```

Exercise - Quoting

Let's say we have the following situation:

```
$ ls
trial line
$ cat trial
This line contains a * character
this line does not
$
```

What happens when entering this command:

```
$ grep * trial
```

?

Exercise - Quoting

Let's say we have the following situation:

```
$ ls
trial line
$ cat trial
This line contains a * character
this line does not
$
```

What happens when entering this command:

```
$ grep * trial
```

?

grep does not find any line in the file, because the shell expands the * to

```
trial line
```

after which the actual command

```
$ grep trial line trial
```

executed

Quoting

```
$ ls
trial  Line
$ grep * trial
$ mv Line line
$ ls
line  trial
$ grep * trial
trial:This line contains a * character
trial:this line does not
trial:This line contains a * character
trial:this line does not
```

Quoting also removes the meaning of spaces as the separator for parameters:

```
$ grep "This line" trial
This line contains a * character
$
```

With the quotes, "This" is the pattern together with "line" to search in the file `trial`. Searching without the quotes, the pattern "This" happens in both the files called `line` and `trial`.

Quoting

When using backticks (this character: `, *not* this: ') for quoting interprets the string as a command. The shell first executes this quoted string as a command and then continues to process the result.

```
$ pwd
/home/bcr/Lectures/Unix/Shell
$ echo pwd
pwd
$ echo `pwd`
/home/bcr/Lectures/Unix/Shell
$
```

In this example, the output of the command `pwd` becomes the parameter for the `echo` command.

Overview

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Introduction to Pipes

When there is no pattern supplied for `grep`, then `grep` will search the input line by line for matches:

```
$ grep abc
Line without pattern    ← User input
Line with pattern abc   ← user input
Line with pattern abc   ← grep output to confirm the search result
^D                      ← Input to end grep (CTRL+D)
$
```

When echoing files to `stdout` supply the `cat` command with files as parameters:

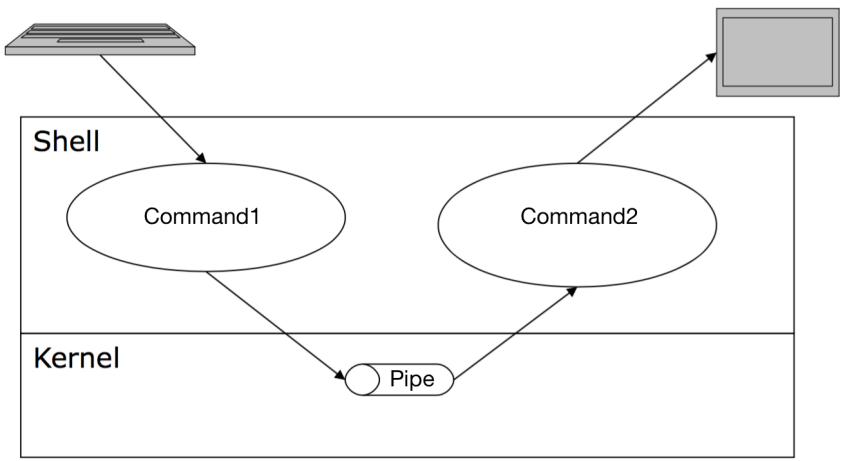
```
$ cat mobile_phone mobile_phone
Anne    0171 3221717
Jack    0172 2393821
Anne    0171 3221717
Jack    0172 2393821
$
```

Pipes

Unix supports the use of filters: programs that read from `stdin` and write to `stdout`. To connect the output of one program to the input of another program, which runs in parallel, then use the pipe operator (|). Using

```
Command 1 | Command 2
```

the output of `Command 1` is send to `Command 2`. Both programs run as separate processes.



Pipes

Example: `date` and `cut` commands

```
$ date
Tue Jan  8 17:18:26 MET 2012
$ date | cut -c12-19
17:18:28
$
```

Count the number of users connected to the system at the moment:

```
$ who | wc -l
37
$
```

Overview

1. Shell Basics
   - Command Completion
   - Moving and Erasing Around The Cursor Position
   - Environment Variables
   - Command History
   - Input and Output Redirection
   - Combining Commands
   - Aliases
   - Background Jobs
   - Wildcards
   - Quoting
   - Pipes
2. Changing the Shell

Changing the Shell

Change the current shell using the following programs, which are basically one and the same: `chpass`, `chfn`, `chsh`, `ypchpass`, `ypchfn`, and `ypchsh`. Requiring the least typing for `chsh`, which is why we show it here in more detail. Without any parameter the standard editor runs (`$EDITOR` environment variable):

```
# chsh
Changing user information for susan.
Shell: /usr/local/bin/zsh
Full Name: User &
Office Location:
Office Phone:
Home Phone:
Other information:
```

The line `Shell:` is changeable. A list of all available shells in the system is located in `/etc/shells` with their full path.
Changing the shell for this user is even faster when passing the shell directly as a parameter:

```
$ chsh -s /bin/csh
Password:
chsh: user information updated
```

Further Information



[FBSDSH] The FreeBSD Documentation Project
The FreeBSD Handbook - 4.9. Shells
http://www.freebsd.org/doc/en_US.ISO8859-1/books/handbook/shells.html



[EXSHELL] The Explainshell website
write down a command-line to see the help text that matches each argument
https://explainshell.com/



[KAM] Shell Tutorial for Absolute Beginners
Test
https://kamila.is/teaching/shell



[EFSH] Effective Shell Tutorial
test
https://effective-shell.com/